Exam

Critical Mass Simulation

Dylan Assinner

May 2026

Introduction

Fission is a well studied process used in nuclear physics for the construction of nuclear power plants and nuclear weapons. One of the elements most commonly used for fission reactions is Uranium. It has two major isotopes: Uranium-238 and Uranium-235. Uranium-238 (^{238}U) can be found in nature and is a radioactive element, producing α-type radiation with a half-life of $4.463 \cdot 10^9$ years [3]. There is also the more radioactive isotope uranium-235 (^{235}U). This isotope is more favorable for fission.
The fission reaction we are going to study is fission using ^{235}U. If we add a neutron to this core, it transitions into the very unstable isotope of Uranium-236. This can then split into two or more lighter elements and an excess of neutrons. There are two prominent reactions here:

$$^1_0\text{n} + {}^{235}_{92}\text{U} \rightarrow {}^{236}_{92}\text{U} \rightarrow {}^{140}_{54}\text{Xe} + {}^{94}_{38}\text{Kr} + 2\ {}^1_0\text{n}$$

$$^1_0\text{n} + {}^{235}_{92}\text{U} \rightarrow {}^{236}_{92}\text{U} \rightarrow {}^{141}_{56}\text{Ba} + {}^{92}_{36}\text{Kr} + 3\ {}^1_0\text{n}$$

In this simulation, we will use the first reaction as a basis.
The intent of the simulation is to calculate the critical mass. This is when the reaction becomes self-sustaining.
In order to explain the concept of critical mass, we need to look at the environment in which the fission occurs. We will assume that there are no possibilities for fission outside our simulated mass (which will consist entirely of 235U and will always follow the first reaction, for simplicity). We also assume that every neutron produced will travel a short random distance. If it is still inside the mass at that point, it will provoke another fission reaction.
The critical mass is thus the mass for which an equal amount of neutrons stay inside the mass as are leaving the object, thus maintaining a constant steady amount of reactions. If the mass is greater than, the critical mass, it will become what we call “supercritical”, meaning that more fission reactions are initiated than originally started. If the mass is lower than the critical mass, the amount of fission reactions will diminish and eventually stop.

1 Implementation

The math for this project will be split into three parts:

1. Calculating the survivor fraction
2. coordinate randomization
3. mass distribution

1.1 calculating the survivor fraction

The survivor fraction is how we measure whether the Uranium has reached the critical mass. It is calculated by dividing the previous "generation" of fission by the current "generation" of fission.
If the survivor fraction (f) is lower than 1, we can conclude that we are below the critical mass, meaning that the reaction will die out.
If the survivor fraction is higher than 1, we can conclude that the reaction has gone supercritical.
By changing the mass (user input), we can try to find the point at which the survivor fraction is 1. To make sure this is accurate, we will allow the user to insert an amount of points (N) that need to be simulated. we will then simulate those N random points inside the volume. this will be our first generation. We will then simulate fission for each of those N points. If we check whether the neutrons are inside the volume, we can calculate the survivor fraction with the formula:

$$f = \frac{N_{\text{inside volume}}}{N_{\text{gen 1}}} \tag{1}$$

To be able to check whether the neutrons are inside the volume, we first need to build a randomizer for the movement of the neutrons inside the simulations.

1.2 coordinate randomization

Using a randomizer to generate random uniformly distributed points, we can calculate the path of the neutrons. If the neutrons have starting coordinates (x_0, y_0, z_0), we can describe their endpoints (x_1, y_1, z_1) using the equations:

$$\begin{cases} x_1 = x_0 + d\ \sin\theta\ \cos\phi \\ y_1 = y_0 + d\ \sin\theta\ \sin\phi \\ z_1 = z_0 + d\ \cos\theta \end{cases} \tag{2}$$

Here, d is the distance the neutron travels and θ and ϕ are the angles of spherical coordinates. We will need to randomize these three variables.

d Is a random variable between 1 and 0. ϕ Is a random variable between 2π and 0. Both of these variables are uniformly distributed.
θ is a bit trickier, for $\cos\theta$ is the uniformly distributed variable. We will thus have to randomize a value between -1 and 1 and use the arccos function to calculate the angle θ.

1.3 mass distribution

For the mass distribution, we have to take into account that we have two possible shapes to simulate: a beam (or cube) and a sphere.

1.3.1 beam

For the beam, we will assume that the mass (M) is equal to the volume (V). We can then change the shape of the beam using a scale factor (S). The influence of the scale factor can be shown with the following equations[1]:

$$M = V = a^2 b = \frac{a^2}{S} = b^3 S^2$$

Here, a is the width and the height of the beam and b is the depth of the beam. If we want to calculate a and b, we can derive following equations[1]:

$$a = \sqrt[3]{M \cdot S} \tag{3}$$

$$b = \sqrt[3]{\frac{M}{S^2}} \tag{4}$$

We can thus see that for $S = 1$, we get a perfect square.
We will assume that this beam is centered around (0, 0, 0). We can then create a starting point randomizer, using a uniform distribution between the following endpoints:

$$\begin{cases} x & \in \left[-\frac{a}{2}, \frac{a}{2}\right] \\ y & \in \left[-\frac{a}{2}, \frac{a}{2}\right] \\ z & \in \left[-\frac{b}{2}, \frac{b}{2}\right] \end{cases}$$

We can also use these conditions for the evaluation to check whether a point is inside the beam object.

1.3.2 sphere

For the sphere, we will also assume that the mass is equal to the volume. For this class, we don't need to implement a scale factor. We do however need to calculate the radius. this can be done by using the formula:

$$V = \frac{4}{3}\pi\, r^3$$

If we then substitute V for M, we can derive the formula for the radius:

$$r = \sqrt[3]{\frac{3}{4}\,M} \tag{5}$$

For the random starting points inside the sphere, we can reuse the code for coordinates of a neutron. Because the distance for the neutrons is between 0 and 1, we can simply multiply this number with the radius to get the desired output.
To evaluate whether a point is inside this sphere, we calculate the distance from the center of the circle (0, 0, 0) and compare this to the radius.

2 Flowcharts

To get a clear view of how the code works, we will use flowcharts of the classes , but first we will explain the way the files are organized:

- **particle.py**
  contains the Particle parent class and the Uranium and Neutron subclasses.
- **shapes.py**
  contains the implementation of the Simulator parent class with the beam (BeamSim) and sphere (SphereSim) subclasses. It also contains an unused Point class.
- **test.py**
  contains the script with the explanation and implementation of the code for the tasks[1].
- **randomizer.py**
  testing file for the randomizer implemented in the Neutron and SphereSim classes.
- **color_picker.md**
  markdown file containing the hexcodes with the colors used in visual simulations.

All these files, along with the files of this report are available through the GitHub repository: `https://gitfront.io/r/Destruckting/vQDpr4HM43Ue/examentaak/`
The flowcharts are split into three, one for the Simulator parent class, as the SphereSim and BeamSim subclasses don't change the working, and one for both the Uranium and Neutron class, as the Particle parent class only adds property protection and string representation.

2.1 legend

The flowcharts were made using the flowchart template on the website `https://mermaid.ai/web/` (Please note that while this website is for an ai agent, this agent was not used, rather the code to chart feature was the reason for its use.)



Figure 1: legend for flowcharts

3 flowcharts

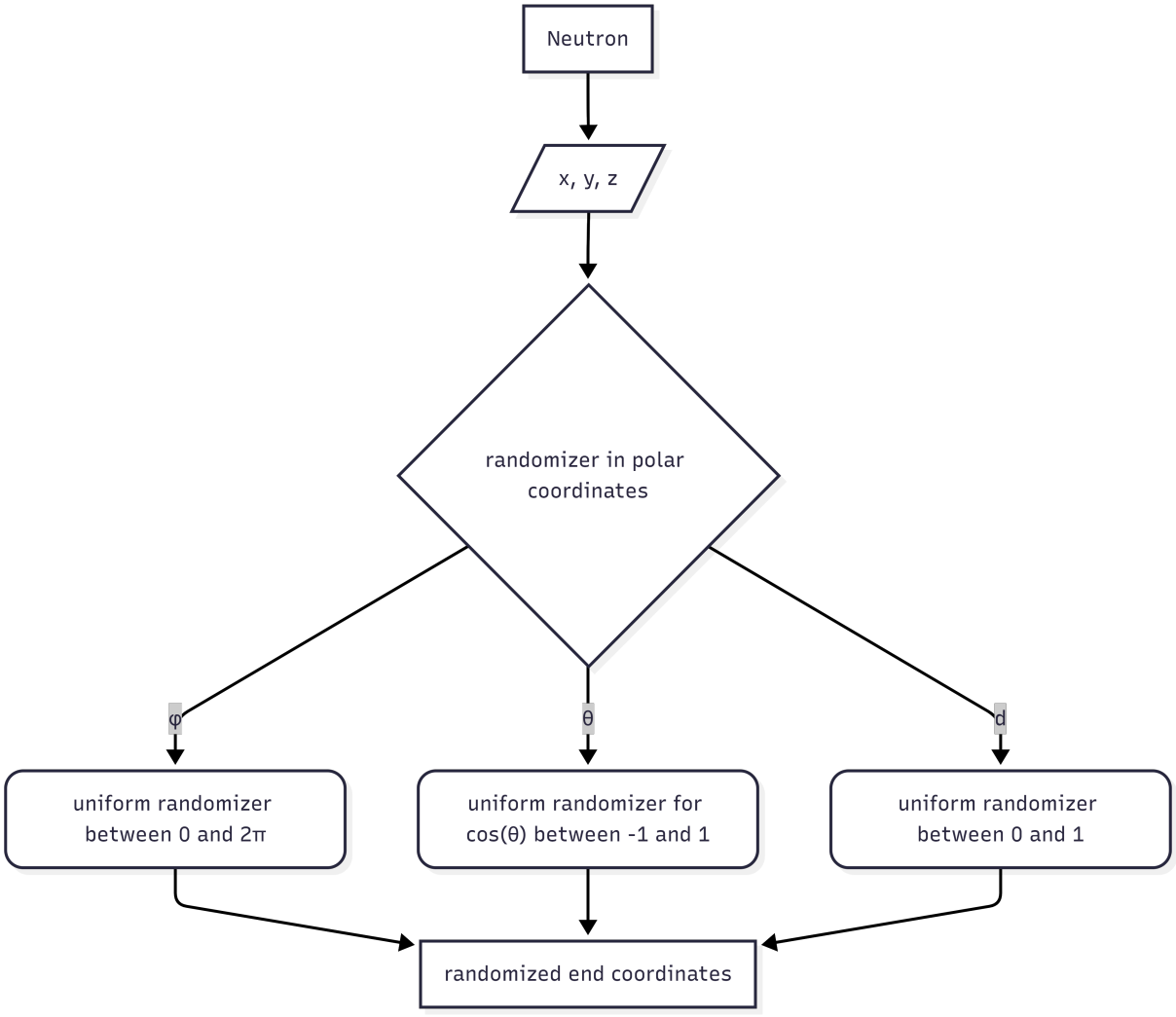


Figure 2: flowchart for Neutron class

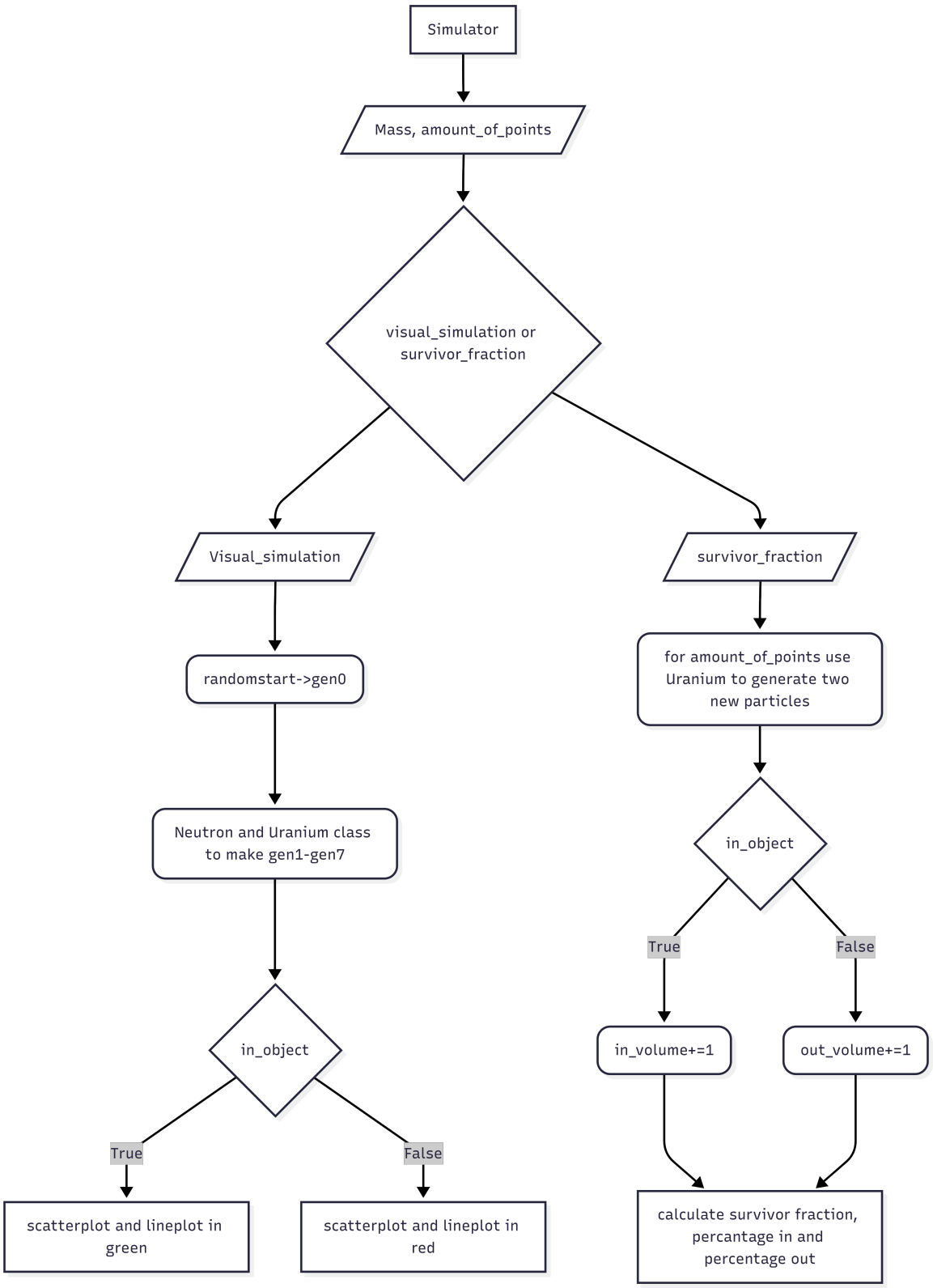


Figure 3: flowchart for Simulator classes

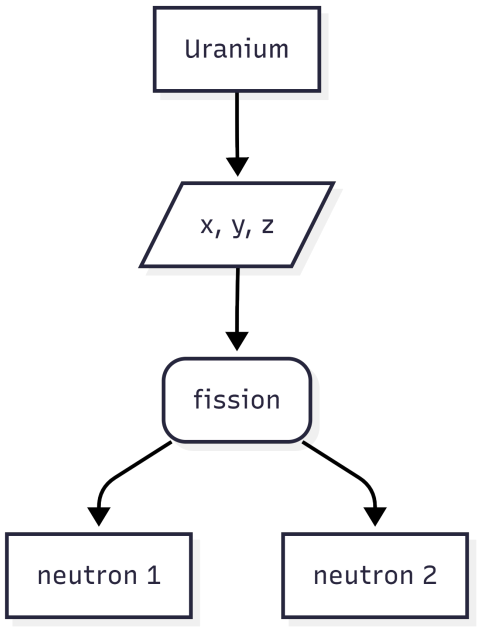


Figure 4: flowchart for Uranium class

4 Behind the code

4.1 Particle class

The Particle class is the parent class off the Neutron class and the Uranium class. It contains property protection for the x, y and z coordinates, along with the `__repr__` and `__str__` functions.

4.2 Neutron class (Figure 2)

The Neutron class inherits the above mentioned. It also contains the randomizers we talked about in section 1.2. We made these randomizers using the `numpy` package. Each variable has its own RNG key to enhance randomness. The Neutron class also contains the property protection for these randomized variables, as well as the calculations for the new endpoint (2).

4.3 Uranium class (Figure 4)

The Uranium class also inherits form the Particle class. The only added feature is a function that uses the x, y and z coordinates from the input as the starting point of two new Neutron class objects.

4.4 Simulator class (Figure 3)

The simulator class is the parent class of the BeamSim and SphereSim classes. All functions are programmed in the Simulator class, the subclasses only add or change certain mechanics. The Simulator class contains two main functionalities: calculating the survivor fraction and a visual simulation.
We will first discuss the functions that both methods use.
The first and most important feature is the property protection. The simulator class takes two inputs, the mass and the amount of points to simulate. These properties also get protected. There are also two private properties: the fig and ax property. These contain the groundwork for a three dimensional `matplotlib` plot. This was necessary to make the inheritance and separate plot function work, but more on this later.
The second method is the `__str__` and `__repr__` functions. These are unique for each of the subclasses.
We can now look at the more specialized methods.
The `in_object(x, y, z)` method checks whether the object is located inside the simulated mass. This always returns True for the Simulator class.
The `randomstart()` method is the randomizer for the starting point. It returns a named tuple with the x, y and z coordinates of the starting point. For the Simulator class, this is always set to (0, 0, 0). Most functions that give multiple outputs use named tuples to make the code easier to work with.
This were all the general features, we can now look at the methods that the user will most likely see.

4.4.1 survivor_fraction()

The `survivor_fraction()` method calculates the survivor fraction by generating a user specified amount of `randomstart()` points. These get used as input in the `Uranium` class. The two Neutron class objects that are made are then checked with `in_object()`.
If the particle is inside the mass, it adds one to the variable `in_volume`. If the particle is outside of the mass, it adds one to the variable `out_volume`.
By dividing `in_volume` by `amount_of_points`, we get the survivor fraction. We can also check the fraction of points stat stayed in the volume or went outside the volume by dividing the corresponding variable by two times `amount_of_points`. This allows us to verify the survivor fraction. These three calculated values are given as output in a named tuple.

4.4.2 visual_simulation()

The `visual_simulation()` function uses the `randomstart()` method to make a random starting point inside the volume (gen0). It then uses the Uranium and Neutron class objects in loops to generate a sequence of fission reactions inside the mass. If a particle fall outside of the object, it won't be used for fission anymore.

The code supports up to seven generations of fission, but it usually doesn't get that far in the plots for the subclasses. If the lists for all generations are made, we will loop over these lists once again using a private `plot_function()` method.
This function is declared as private by appending `__` to the name of the function. This `plot_function()` takes a list as input for the points to plot, along with the type of marker you want to use in the scatterplot and the color for the objects inside the mass and outside the mass.
For a complete list of all colors, please refer to *color_picker.md* for the list of hexcodes for each generation. We used the standard dot as marker, but other types like numbers are also supported (see `matplotlib` documentation). When running the program, you can follow the way the fission reactions occurred by following the color from dark to light. The random starting point is plotted in black. This is also made easier by not only drawing the scatterplot of the points, but also drawing a line in the color of the endpoint to the new marker.

4.5 BeamSim class

Because it inherits almost all functions from the Simulator class, we will quickly go over the added or changed functions. As seen in section 1.3.1, the simulation of a beam shaped uranium object requires an extra scale input. We then calculate the width and the depth using the formulae mentioned in that section.
As mentioned in the previous section, the `__repr__` and `__str__` functions are unique for each of the subclasses.
For the `randomstart()` function, we use three separate RNG keys for each of the coordinates. This assures no connection between the variables. Every variable then gets a random uniformly distributed value between -0.5 and 0.5, which we mutliply with the corresponding value for width or depth. This function then returns a named tuple.
For the check whether a particle is inside the object (`in_object(x, y, z)`), we use the conditions mentioned in section 1.3.1 inside a `bool()` function. This standard function returns true if all the conditions inside the brackets are met and false if not.
There are no changes made to the `survivor_fraction()` function in any of the subclasses.
For the `visual_simulation()` method, we added lines to display the beam in the plot window.

4.6 SphereSim class

This function takes no extra input arguments. It uses the math in section 1.3.2.
For the starting point, we reuse the code from the Neutron class randomizer, with the calculated radius of the sphere multiplied by the distance modifier.
The check `in_object()` is also programmed using the `bool()` function, but this time we check whether the disance from the center is smaller than or equal to the radius.
The reason why we needed to add the fig and ax variables as inherent properties of the simulators is not only to make working with a separate plotting function easier, but also to

make the use of `matplotlib.patches` possible. The use of this subpackage of `matplotlib` makes the drawing of smooth circles possible. To get a clear representation of the sphere in the visual simulation, we draw a circle of the calculated radius along the x, y and z axis.

4.7 Point class

The Point class is a function that was implemented with the idea of a more generalized placement of the objects. This however was deemed unnecessary and the input option was thus removed in the Simulator classes. The class is however still used with a standard position at (0, 0, 0). The Point class takes an x, y and z coordinate as input. It has the `__repr__`, `__str__`, `__add__` and `__sub__` functions defined, along with a function to return the point in spherical coordinates.

4.8 test.py

Now that all the classes and their methods are explained, we will talk a little bit about the `test.py` file.
Because the figure and axes of the `matplotlib` plot are inherent attributes of the simulator classes, plot windows are made with each simulator object. This means that these need to be cleared from the memory to avoid excess memory usage. To do this, we defined a small function that plots the windows and closes them in rapid succession, to allow space for new simulations.
To make the execution of the tasks[1] easier, we used the tabulate package to make easily interpretable tables.

4.9 code optimization

The code in all the files was optimized using `pylint` and `pycodestyle`, assuring that the code was clear and readable.

5 results

Now that the way the code works is explained, we will show the results of working with the code.

5.1 visual simulations

First, we will showcase the visual simulations using a Simulator class object and a BeamSim and SphereSim object.

5.1.1 Simulator class object

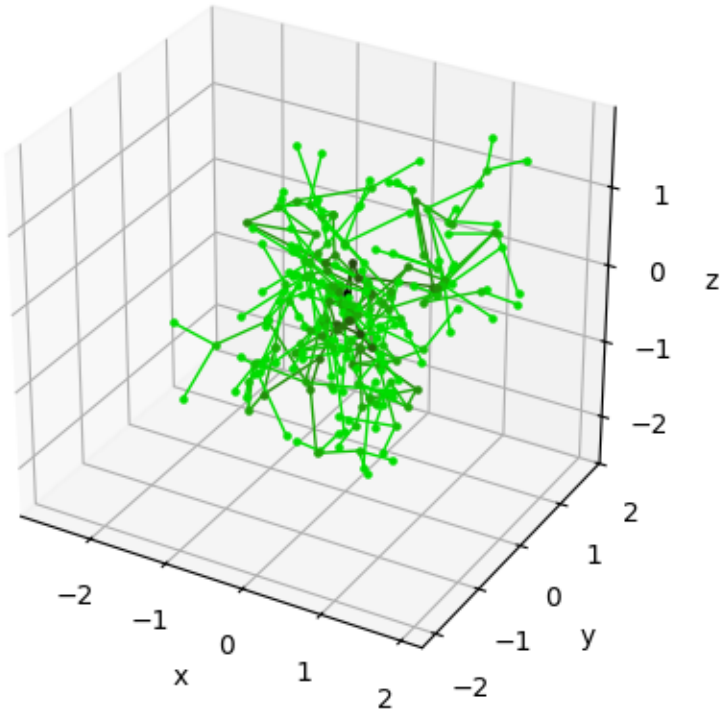


Figure 5: visual simulation Simulator class object

The dots are all green here, because the Simulator class always returns True for `in_object (x, y, z)`. Note that the center, where the starting point (0, 0, 0) is contains a lot more darker points, whereas the points towards the edges are more of the lighter green, signifying later generations.
All visual simulations are taken from the testing code at the bottom of *shapes.py*.

5.1.2 BeamSim class object

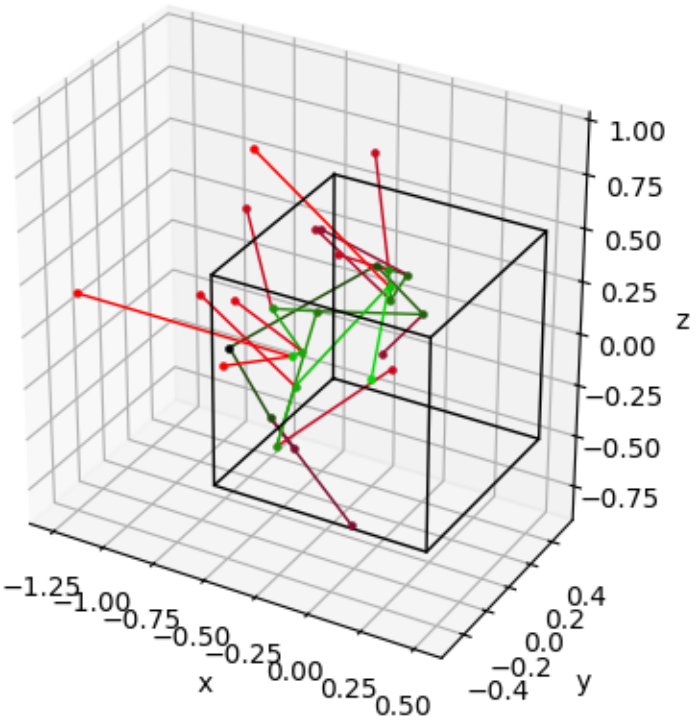


Figure 6: visual simulation BeamSim class object

This plot has a mass of 1 and a scale of 1. Thus we get a square. Note that the amount of points is considerably less than the Simulator object. It is also clear that the red points fall outside the volume, while the green lines stay within the mass. Due to the smaller amount of points, it is also considerably easier to follow the order of the points.

5.1.3 SphereSim class object

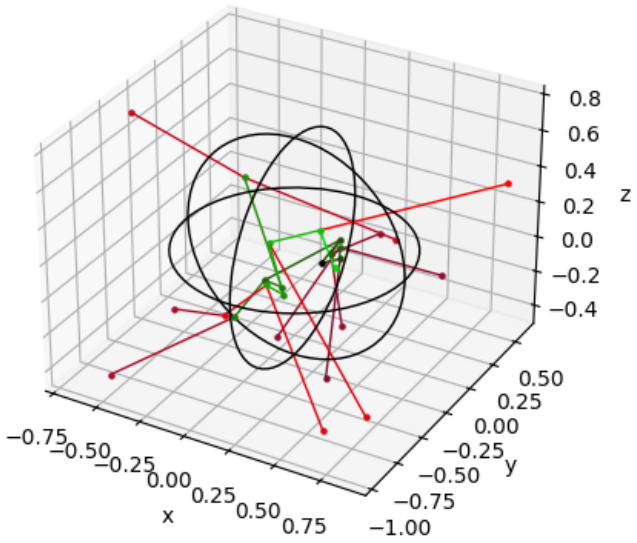


Figure 7: visual simulation SphereSim class object

The SphereSim can sometimes be a hard class to get a nice plots, as the points sometimes leave the object as soon as the first generation.

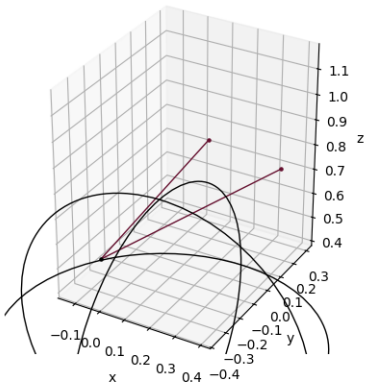


Figure 8: failed visual simulation SphereSim class object

5.2 first task: finding the critical mass of a cube

The critical mass will be calculated in three steps. First, we will find a general vicinity for the solution by using 100 points to start with. Then we will try this simulation with a smaller range using 1000 simulated points. Finally we will try an even smaller range with 10000 simulated points to estimate a margin of error. All these steps were done multiple times before continuing the code, to make sure the estimates were accurate enough, as the results tend to fluctuate.
For the critical mass of a cube we used scale factor 1 and found a critical mass around a mass of 1.5 with an error margin of 0.3.

5.3 second task: finding the critical mass of a beam with scale 0.5 and scale 1.5

For the second task, we used the same methods, but now we used a lot more different scale factors.
We started with a scale of 1.5, for which we found a critical mass of 1 with an error of 0.2. We then tried a scale of 0.5, but this never seemed to give us a critical mass, even for a mass of 1000. We then experimented with a scale of 2 and 0.9, to get a better understanding of how the critical mass changed depending on the scale. We then found the following data:

scale	critical mass
2	0.5
1.5	1
1	1.5
0.9	3
0.5	undefined

Table 1: critical mass for different scales

From this table, we can estimate that it follows an exponential or power curve, but without more points it will be hard to get an accurate fit on the data.

5.4 third task: finding the critical mass of a sphere

For the final task, we had to find the critical mass of a sphere. We found that the critical mass was 0.875 with an error margin of 0.125.

6 Conclusion

References

[1] Nick Van Remortel and Federico De Lillo. *Examen Eerste Zittijd 2026 Inleiding programmeren voor fysici*. 2026.

[2] Wikipedia contributors. *Uranium*. Apr. 24, 2026. URL: `https://en.wikipedia.org/wiki/Uranium#`.

[3] Wikipedia contributors. *Uranium-238*. Apr. 6, 2026. URL: `https://en.wikipedia.org/wiki/Uranium-238`.